

Chapter 6: HCI in SW process

Software engineering provides a way to **understand the structure of the design process**, which can then be assessed for its effectiveness in interactive system design.

Usability engineering promotes **explicit, measurable criteria for judging a product's success in terms of its usability**.

Design rationale **records design decisions** and the **context in which they were made**

6.1 Introduction

Usability concerns affect **every stage** of the software life cycle.

› Four Big Ideas

- 1) **Software Engineering: Provides structure for the design process.**
- 2) **Usability Engineering: Uses measurable usability goals.**
- 3) **Iterative Design: Improves systems through repeated evaluation and redesign.**
- 4) **Design Rationale: Records design decisions and reasons behind them.**

6.2 The Software Life Cycle

- **Customer** — **Person/group** requesting the system → interacts w design team
- **Designer** — person/group who provides the product.
- **User** — **Person/group** actually using the system.

6.2.1 Waterfall Model

each activity naturally leads into the next, like water cascading down.

Requirements → Architectural → Detailed → Coding → Integration → Maintenance

Activity	What happens
1. Requirements specification	Define WHAT the system should do. • Identify functions required. • Understand users and environment. • Written initially in natural language.
2. Architectural design	Define HOW the system is organized. • Split the system into components. • Define component interactions. • Consider functional and non-functional requirements. Functional Requirements: Services the system performs. Non-Functional Requirements • Reliability + Efficiency + Safety + Usability
3. Detailed design	Refine each component. Make it ready for implementation. Choose designs that satisfy non-functional requirements.
4. Coding & unit testing	Convert design into code & Test individual components. Automatic code & test generation.
5. Integration & testing	• Combine components. • Test complete system. • Acceptance testing by customer. • Certification may be required. • Release system
6. Operation & maintenance	• Fix errors after release. • Add new requirements. • Longest phase of system life.

6.2.2 Validation and Verification

Validation → "Are we building the right thing?"

- Checks customer needs.
- Focuses on requirements.

Verification → "Are we building the thing right?"

- Checks correctness and consistency.
 - Focuses on design quality.
-

Formal Proof: Fully mathematical proof.

Rigorous Proof: Mostly mathematical but omits some details.

The difference is simply the amount of detail the prover leaves out while still maintaining acceptance. Time constraints and economic implications dictate which proofs are done formally, rigorously, or not at all.

> Formality Gap

difference between:

- **Real-world user requirements** (what users actually need)
- **Formal design descriptions** (technical models, specifications, diagrams).

Problem

- Real-world requirements are often vague and difficult to express exactly.
- Formal designs are precise and technical.
- Some meaning may be lost when converting requirements into formal specifications.

- Therefore, it is impossible to guarantee that the design perfectly matches user needs (**perfect validation is impossible**).

Solution

- **Use domain experts.**
 - Experts understand the real-world environment and user needs.
 - They can **check whether the design reflects actual requirements**.
- **Use understandable design notations.**
 - Use diagrams, models, and documentation that stakeholders can easily understand.
 - This helps users and experts verify the design more accurately

6.2.3 Management and Contractual Issues

› **Technical Perspective:** **Focuses on activities and design tasks.**

› **Managerial Perspective:** **Focuses on time, cost, staffing, training, and market needs.**

› Stages vs Phases

Stages : **Technical activities.**

Phases: **Management and documentation milestones.**

› Contractual Obligation

- Customer and designer approve documents.
- **Signed requirements** become commitments.

Main Problem: Requirements are often unknown until users actually use the system.

Risk: Requirements become Too vague or Too restrictive

6.2.4 Interactive Systems and the Software Life Cycle

› Why Traditional Models Are Insufficient

- Created for batch-processing systems.
- Modern systems are highly interactive.

› Why Requirements Cannot Be Fully Known Early

- Human behavior cannot be perfectly predicted.
- Users must interact with realistic systems.
- New tasks emerge after users start using the system.
- Users find uses never planned by designers.
- Traditional design methods ignore user perspective.

6.3 Usability Engineering

Approach that introduces explicit usability goals into development.

Goal: Know exactly how usability will be measured.

Usability Specification

For each usability attribute define:

Item	Meaning
Attribute	What is being measured.

Measuring concept	Specific aspect being evaluated.
Measuring method	how it is measured
Now level	Current performance.
Worst case	Lowest acceptable measurement — the acceptable/unacceptable boundary. Minimum acceptable value.
Planned level	Target value
Best case	Best possible given current tools/technology. Ideal value.

CASE STUDY: VCR BACKWARD RECOVERABILITY SPEC

Old VCR systems were notoriously hard to program and had no undo mechanism—if a user made one mistake, they had to restart the whole sequence from scratch (Now Level).

Usability Spec for new VCR:

- Attribute: Backward recoverability (undoing a programming error).
- Method: Number of explicit user actions required to correct a mistake.
- Worst Case: As many actions as it takes to program the mistake again.
- Planned Level: A maximum of two actions (click Back → confirm).
- Best Case: One explicit cancel/undo action.

Measurement Criteria Examples

- Task completion time
- Success rate
- Error rate

- Help usage
- User satisfaction
- User frustration
- Workarounds used

Ways to Set Measurement Levels

- Existing system
- Competitor systems
- Manual process
- Absolute scale
- Prototype
- Previous user performance

ISO 9241 Usability Metrics

- **Effectiveness:** Accuracy and completion of goals.
- **Efficiency:** Resources/time used.
- **Satisfaction:** User comfort and acceptance.

Usability objective	Effectiveness	Efficiency	Satisfaction
Suitability for the task	% of goals achieved	Time to complete a task	Rating scale for satisfaction
Appropriate for trained users	No. of power features used	Relative efficiency vs. an expert user	Rating scale for satisfaction with power features
Learnability	% of functions learned	Time to learn criterion	Rating scale for ease of learning
Error tolerance	% of errors corrected successfully	Time spent correcting errors	Rating scale for error handling

6.3.1 Problems with Usability Engineering

- **Problem 1:** Metrics require details unavailable early in design.

- Problem 2: **Meeting metrics** does **not guarantee true usability**.

6.4 Iterative Design and Prototyping

Iterative Design: **Build** → **Evaluate** → **Improve** → **Repeat**

Purpose: Handle incomplete requirements.

- iterative design uses **prototypes** — artifacts that **simulate or animate some, but not all, features** of the intended system

Three main approaches to prototyping

Approach	Description
Throw-away	The prototype is built, tested with users to gather requirements or clarify ideas, and then completely discarded. The final system is rebuilt from scratch using the insights gained. • Prototype discarded. • Knowledge retained.
Incremental	The system is partitioned into independent functional modules. Each module is prototyped, tested, and added piece-by-piece to form the final combined system. • System built in pieces. • Components added gradually.
Evolutionary	• Prototype becomes final product. • The prototype is NOT discarded • Continuously refined.

Management problems with prototyping (Sommerville)

- **Time** → **Prototypes require additional effort**.
- **Planning** → **Difficult to estimate costs**.
- **Non-Functional Features** → **Safety and reliability often missing**.

- **Contracts** → **Prototypes cannot replace formal documentation.**

6.4.1 Prototyping Techniques

Storyboards

- **Drawings/screens** of interface.
- **No functionality.**
- **Fast and cheap.**

Limited Functionality Simulations

- **Simulate some real functions.**
- **Allow user interaction.**
- (e.g. clicking a "Like" button changes the icon color, but the database backend isn't connected)

Wizard of Oz Technique

- Human **secretly performs missing system functions.**
- User believes system is complete.

High-Level Programming Support

- **Specialized languages for rapid interface development.**

UIMS (User Interface Management System)

Purpose → **Separates interface from application functionality.**

Benefit → **Interface can be tested independently**

6.4.2 A Warning About Iterative Design

Problem 1: Design inertia / early bad decisions: Early bad decisions may persist.

Problem 2: Treating the symptom, not the cause: Fixing visible problems instead of root causes.

6.5 Design Rationale

Documentation explaining why design decisions were made.

- Occurs Throughout the entire software life cycle.

Benefits

- **Communication:** Explains decisions to future designers.
- **Knowledge Reuse:** Reuse successful solutions.
- **Better Deliberation:** Encourages careful thinking.

Why Important in HCI

- **No Single Best Design:** Trade-offs are unavoidable.
- **Huge Design Space:** Many possible solutions.
- **Context Dependency:** Usability depends on the environment and users.

Factors Distinguishing Techniques

- **Degree of Intrusion:** How much it affects design activities.
- **Cost:** Effort to create and use.
- **Tool Support:** Ability to manage large rationale repositories.

Types of Design Rationale

6.5.1 Process–Oriented Design Rationale

- Historical record of decisions
- Records **chronological history** of decisions

Technique: **IBIS [issue-based information system]**

- **Issue:** Problem being discussed.
- **Position:** Proposed/possible solution.
- **Argument:** **Supports** or **opposes a position.**
- **Sub-Issue:** New related issue.

gIBIS → **Graphical version** of IBIS.

PHI → **Extended issue relationships.**

6.5.2 Design Space Analysis (Structure–Oriented)

- **Structure of design alternatives.**
- Maps **all possible design choices** as a static space

Technique: **QOC (Questions, Options, Criteria)**

- **Question:** Design issue.
- **Option:** Possible solutions
- **Criteria:** Evaluation factors.

Mapping: Options are linked to Criteria with **solid lines** (positive support/good match) or **dashed lines** (negative support/poor match). A **highlighted box** denotes the **chosen option**.

Advantages

- Easier knowledge reuse.

Disadvantages

- Additional documentation effort.



Exam Tip: Limitations of QOC

Students often mistake QOC for a decision generator. In reality, QOC has strict limitations:

1. It does *not* show which criterion is more important (no weights).
2. It does *not* always yield a single "winner" option.

DRL (Decision Representation Language)

- **Decision Problem:** Question
- **Alternatives:** Options
- **Goals:** Criteria

Advantage

- Supports automatic dependency tracking.

6.5.3 Psychological Design Rationale

Devs: Carroll & Rosson

Focus → Psychological consequences of design decisions.

- Records effects on users.

- **mainly process-oriented**

Task-Artifact Cycle

- **Step 1:** Users perform tasks.
- **Step 2:** System is designed.
- **Step 3:** Users discover new tasks.
- **Step 4:** New tasks generate new requirements.

Cycle repeats.

User Questions

- **What can I do?** Available functions.
- **How does it work?** Understand behavior.
- **How can I do this?** Learn task procedures.

Claims Analysis

- **Positive Claims:** Benefits for users.
- **Negative Claims:** Problems introduced.

Purpose: Improve future designs.

Worked Exercise (from the chapter)

Q: What is the distinction between a process-oriented and a structure-oriented design rationale technique? Would you classify psychological design rationale as process or structure oriented? Why?

A (summary). The distinction is what information the rationale captures.

Process-oriented rationale records a historically accurate description of a team making a decision on an issue — an activity concurrent with design.

Structure-oriented rationale is less concerned with historical evolution and more

with the **conclusions of the design activity**, so it can be done post hoc and reflectively.

Psychological design rationale supports the task–artifact cycle: **designers record the tasks the system should support, build to support them, suggest scenarios used to observe new users**, and the observations provide the actual rationale for that version, gauging the design's assumptions against real use to justify the design or suggest improvements.

It is **mainly process-oriented**: a claims analysis captures what designers assumed at one point in time and compares it with actual use, so the history matters. The discipline requires performing the claims analysis during the actual design activity, not as a post hoc reconstruction.